

Instruction Name	Type	Explanation	Executed On	Pseudo-code	PC	Where is it used	OPCODE[6:0]	ID
add.s	R-Int	Addition	Scalar Core	rd = rs1 + rs2	PC = PC + 20	Loop Indexing	0000001	1
sub.s	R-Int	Subtraction	Scalar Core	rd = rs1 - rs2	PC = PC + 20	Loop Indexing	0000010	2
mul.s	R-Int	Multiplication	Scalar Core	rd = rs1 * rs2	PC = PC + 20	Loop Indexing	0000011	3
div.s	R-Int	Division	Scalar Core	rd = rs1 / rs2	PC = PC + 20	Count Number of Tiles	0000100	4
mod.s	R-Int	Modulous	Scalar Core	rd = rs1 % rs2	PC = PC + 20	Non-powers of 2 tiles	0000101	5
or.s	R-Int	Bitwise Or	Scalar Core	rd = rs1 rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions	0000110	6
and.s	R-Int	Bitwise And	Scalar Core	rd = rs1 & rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions	0000111	7
xor.s	R-Int	Bitwise Xor	Scalar Core	rd = rs1 ^ rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions	0001000	8
sll.s	R-Int	Shift left logical	Scalar Core	rd = rs1 << rs2	PC = PC + 20		0001001	9
srl.s	R-Int	Shift Right Logical	Scalar Core	rd = rs1 >> rs2	PC = PC + 20		0001010	10
sra.s	R-Int	Shift Right Arith	Scalar Core	rd = rs1 >> rs2	PC = PC + 20		0001011	11
slt.s	R-Int	Set Less Than	Scalar Core	rd = rs1 < rs2	PC = PC + 20		0001100	12
sltu.s	R-Int	Set Less Than (U)	Scalar Core	rd = u(rs1) < u(rs2)	PC = PC + 20		0001101	13
bfs.s	R-Int	bFloat to scalar	Scalar Core	rd = Int(rs1)	PC = PC + 20	Convert Scalar to BF16, makes compiler's life really easy.	0001110	14
add.bf	R-Float	Add bFloat	Scalar Core	rd = rs1 + rs2	PC = PC + 20		0001111	15
sub.bf	R-Float	Sub bFloat	Scalar Core	rd = rs1 - rs2	PC = PC + 20		0010000	16
mul.bf	R-Float	Mult bFloat	Scalar Core	rd = rs1 * rs2	PC = PC + 20		0010001	17
rcp.bf	R-Float	Reciprocal bFloat	Scalar Core	rd = 1 / rs1	PC = PC + 20	all vector divisions in the kernels can be simplified 1/scalar * elementwise multiply.	0010010	18
slt.bf	R-Float	Set Less Than BF16	Scalar Core	rd = rs1 < rs2	PC = PC + 20		0010011	19
sqr.bf	R-Float	Square-root bf16	Scalar Core	rd = sqrt(rs1)	PC = PC + 20	sqrt(dk)	0010100	20
stb.f	R-Float	scalar to bFloat	Scalar Core	rd = Float(rs1)	PC = PC + 20	Convert BF16 to Scalar, makes compiler easier to build.	0010101	21
add.i	I-Type-Int	Addition Immediate	Scalar Core	rd = rs1 + imm	PC = PC + 20		0010110	22
sub.i	I-Type-Int	Subtraction Immediate	Scalar Core	rd = rs1 - imm	PC = PC + 20		0010111	23
mul.i	I-Type-Int	Multiplication Immediate	Scalar Core	rd = rs1 * imm	PC = PC + 20		0011000	24
div.i	I-Type-Int	Division Immediate	Scalar Core	rd = rs1 / imm	PC = PC + 20		0011001	25
mod.i	I-Type-Int	Modulous Immediate	Scalar Core	rd = rs1 % imm	PC = PC + 20		0011010	26
or.i	I-Type-Int	Bitwise Or Immediate	Scalar Core	rd = rs1 imm	PC = PC + 20		0011011	27
and.i	I-Type-Int	Bitwise And Immediate	Scalar Core	rd = rs1 & imm	PC = PC + 20		0011100	28
xor.i	I-Type-Int	Bitwise Xor Immediate	Scalar Core	rd = rs1 ^ imm	PC = PC + 20		0011101	29
slli.s	I-Type-Int	Shift left logical Immediate	Scalar Core	rd = rs1 << (imm & 0x1F)	PC = PC + 20		0011110	30
srl.i	I-Type-Int	Shift Right Logical Immediate	Scalar Core	rd = rs1 >> (imm & 0x1F)	PC = PC + 20		0011111	31
srai.s	I-Type-Int	Shift Right Arith Immediate	Scalar Core	rd = rs1 >> (imm & 0x1F)	PC = PC + 20		0100000	32
jalr.s	I-Type-Int	Jump and Link Return	Scalar Core	rd = PC + 20;	PC = rs1 + imm	Function Call for Software Libraries	0101110	46
slt.i	I-Type-Int	Set Less Than Immediate	Scalar Core	rd = rs1 < imm	PC = PC + 20		0100001	33
sltu.i	I-Type-Int	Set Less Than (U) Immediate	Scalar Core	rd = u(rs1) < u(imm)	PC = PC + 20		0100010	34
beq.s	B-type	Branch Equal To (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 == rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0100011	35
bne.s	B-type	Branch Not Equal To (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 != rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0100100	36
blt.s	B-type	Branch Less Than (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 < rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0100101	37
bge.s	B-type	Branch Greater Than or Equal to (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 >= rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0100110	38
bgt.s	B-type	Branch Greater Than (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 > rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0100111	39
ble.s	B-type	Branch Less Than or Equal to (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 <= rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.	0101000	40
lw.s	M-type	Load Word	Scalar Core	rd = MEM[rs1 + sign_ext(imm12)]	PC = PC + 20	Bookkeeping logic.	0101001	41
sw.s	M-type	Store Word	Scalar Core	MEM[rs1 + sign_ext(imm12)] = rd	PC = PC + 20	Bookkeeping logic.	0101010	42
lhw.s	M-type	Load Half-Word	Scalar Core	rd[15:0] = MEM[rs1 + sign_ext(imm12)][15:0]	PC = PC + 20	Handling bf16 scalar reads, makes compiler simple.	0101011	43
shw.s	M-type	Store Half-Word	Scalar Core	MEM[rs1 + sign_ext(imm12)][15:0] = rd[15:0]	PC = PC + 20	Handling bf16 scalar reads, makes compiler simple.	0101100	44
jal	MI-Type	Jump and Link	Scalar Core	rd = PC + 20;	PC += imm	Function Call for Software Libraries	0101101	45
li.s	MI-Type	Load Immediate (Pseudo instruction)	Scalar Core	lui and then addi	PC = PC + 20	Assemble RS3 for the SDMA and other instructions	0101111	47

lui.s	MI-Type	Load Upper Immediate	Scalar Core	rd[31:7] = imm[24:0]	PC = PC + 20	Assemble RS3 for the SDMA and other instructions	0110000	48
nop.s	R-Int	No Op	Scalar Core	no op	PC = PC + 20	Padding where needed	0110001	49
halt.s	R-Int	Halt	Scalar Core	halt	PC = PC + 20	Halting is a important constraint!	0110010	50
add.vv	Vector Vector	Elementwise Vector Add	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] + vs2[i];	PC = PC + 20	Partial Sum Accumulation	0110011	51
sub.vv	Vector Vector	Elementwise Vector Subtract	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] - vs2[i];	PC = PC + 20	Couldn't find use, but virtually no extra hardware to support.	0110100	52
mul.vv	Vector Vector	Elementwise Vector Multiply	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] * vs2[i];	PC = PC + 20	Could be used in a sequence to compute the hadamard product. Elementwise multiply here, and then the reduction sum is used.	0110101	53
gemm.vv	Vector Vector	Issue Vector for GeMM on Systolic Array	Vector Core (Executed on Systolic Array).	vd = vs1 * weights_sys_array;	PC = PC + 20	Issue all GeMMs to the systolic array	0110110	54
expi.vi	Vector Immediate	Elementwise e^x	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = exp(vs1[i]);	PC = PC + 20	Elementwise Expontial needed for SoftMax	0110111	55
lw.vi	Vector Immediate	Load a vector into systolic array as weights	Vector Core (Executed on Systolic Array).	weights_sys_array.push(vs1[i])	PC = PC + 20	Load weights into the systolic array	0111000	56
rsum.vi	Vector Immediate	Reduction: sum	Vector Core	result = SUM(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	Reduction for SoftMax	0111001	57
rmin.vi	Vector Immediate	Reduction: min	Vector Core	result = MIN(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	Couldn't find use, but no extra hardware to support.	0111010	58
rmax.vi	Vector Immediate	Reduction: max	Vector Core	result = MAX(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	MaxPool	0111011	59
mgt.mvv	Vector Mask Vector	Mask Greater Than (Vector Vector)	Vector Core			Top-k choices, it might be useful for that.	0111100	60
mlt.mvv	Vector Mask Vector	Mask Less Than (Vector Vector)	Vector Core			Top-k choices, it might be useful for that.	0111101	61
meq.mvv	Vector Mask Vector	Mask Equal to (Vector Vector)	Vector Core			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there.	0111110	62
mneq.mvv	Vector Mask Vector	Mask Not Equal to (Vector Vector)	Vector Core			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there.	0111111	63
mgt.mvs	Vector Mask Scalar	Mask Greater Than (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			ReLU, LeakyReLU, odd sized matrices where the size is a scalar.	1000000	64
mlt.mvs	Vector Mask Scalar	Mask Less Than (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			ReLU, LeakyReLU, odd sized matrices where the size is a scalar.	1000001	65
meq.mvs	Vector Mask Scalar	Mask Equal to (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there. We might want to use this to check for Infinty maybe?	1000010	66

mneq.mvs	Vector Mask Scalar	Mask Not Equal to (Vector Scalar)	Vector Core (Scalar Value Broadcasted)		Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point.	1000011	67
vreg.ld	Vector Memory	Vector Load from Scratchpad			All programs that use the vector core need to load data to that.	1000100	68
vreg.st	Vector Memory	Vector Store to Scratchpad			All programs that use the vector core need to load data to that.	1000101	69
scpad.ld	SDMA	Scratchpad Load from DRAM			All tensors are loaded to the scpad.	1000110	70
scpad.st	SDMA	Scratchpad Store to DRAM			All tensors are loaded to the scpad.	1000111	71
vmov.vts		Move 1 element of a vector to scalar			Used in SoftMax since division (reciprocal) is only in the scalar core. It has to be moved and then broadcast back.	1001000	72
mv.mts		Move Selected Vector Mask to Scalar Core			Mask based on vector length than other variables.	1001001	73
mv.stm		Move Mask from Scalar core to Vector Mask File			Mask based on vector length than other variables.	1001010	74
add.vs	Vector Scalar Arithmetic	Add Scalar to all vector elements	Vector Core (Scalar Value Broadcasted)		Couldn't find use, but virtually no extra hardware to support.	1001011	75
sub.vs	Vector Scalar Arithmetic	Subtract Scalar from all vector elements	Vector Core (Scalar Value Broadcasted)		Couldn't find use, but virtually no extra hardware to support.	1001100	76
mul.vs	Vector Scalar Arithmetic	Mult Scalar to all vector elements	Vector Core (Scalar Value Broadcasted)		Softmax uses this.	1001101	77